



MODUL PERKULIAHAN

SISTEM KENDALI CERDAS

Kode Mata kuliah W132500026

Modul 5

Pemodelan dan Simulasi Sistem

Abstrak

Simulasi memungkinkan pengujian skenario normal, ekstrem, dan gagal sebelum controller menyentuh plant nyata. Hasilnya hanya dapat dipercaya jika model, parameter, kondisi awal, solver, dan metrik validasi dinyatakan dengan jelas.

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 3.1 — Menyusun eksperimen simulasi, memilih solver, dan memvalidasi model



Modul

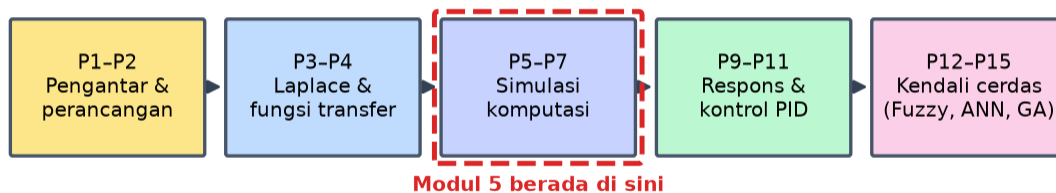
Interaktif:

<https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Sistem-Kendali-Cerdas/Modul/Modul-5.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk membaca seluruh materi tanpa perlu NIM dan PIN. Untuk mengerjakan Tugas dan Forum, masuk sebagai Mahasiswa memakai NIM dan PIN.

PENDAHULUAN

Simulasi memungkinkan pengujian skenario normal, ekstrem, dan gagal sebelum controller menyentuh plant nyata. Hasilnya hanya dapat dipercaya jika model, parameter, kondisi awal, solver, dan metrik validasi dinyatakan dengan jelas. Gambar 1 mengilustrasikan gagasan ini.

Alur Mata Kuliah Sistem Kendali Cerdas



Gambar 1. Posisi Pertemuan 5 (Pemodelan dan Simulasi Sistem Kontrol) dalam alur mata kuliah Sistem Kendali Cerdas.

Modul ini disusun berlapis: pembahasan konsep, penurunan rumus yang dipakai, satu contoh terselesaikan, catatan salah kaprah yang sering terjadi, daftar periksa, lalu tugas terstruktur berupa sepuluh soal pilihan ganda dan lima belas soal komputasi. Versi interaktifnya menilai jawaban secara otomatis di server, sedangkan berkas ini dipakai untuk membaca dan mengerjakan secara luring.

Capaian Pembelajaran (Sub-CPMK)

- Sub-CPMK 3.1 — Menyusun eksperimen simulasi, memilih solver, dan memvalidasi model

SIMULASI SEBAGAI EKSPERIMEN, BUKAN SEKADAR GAMBAR

Simulasi yang berguna dirancang seperti eksperimen: ada pertanyaan yang hendak dijawab, ada besaran yang divariasikan, ada besaran yang dijaga tetap, dan ada kriteria untuk menyatakan hasilnya bermakna. Tanpa pertanyaan yang jelas, simulasi hanya menghasilkan kurva yang enak dipandang namun tidak menuntun keputusan apa pun.

Setiap eksperimen simulasi perlu menyebutkan enam hal: model beserta asumsinya, nilai seluruh parameter, kondisi awal, jenis masukan uji, solver dan langkah waktu, serta metrik yang akan dibandingkan. Enam hal itu pula yang memungkinkan orang lain mengulang hasil dan memperoleh angka yang sama.

Masukan uji dipilih sesuai sifat yang ingin diperiksa. Step menguji respons terhadap perubahan mendadak dan memperlihatkan overshoot serta waktu menetap. Ramp menguji kemampuan mengikuti target yang bergerak dan memperlihatkan error kecepatan. Sinus pada beberapa frekuensi memperlihatkan bandwidth dan penguatan. Impuls memperlihatkan respons alami sistem secara langsung.

Membandingkan hasil simulasi dengan data pengukuran menuntut kehati-hatian pada titik kerja. Model linier hanya berlaku di sekitar titik kerja tempat ia diturunkan, sehingga membandingkan simulasi pada beban ringan dengan pengukuran pada beban penuh akan memberi selisih yang tampak seperti kesalahan model padahal hanya perbedaan kondisi.

eksperimen = {model, parameter, kondisi awal, masukan, solver, metrik}

Ruang Keadaan dan Fungsi Transfer: Dua Sudut Pandang

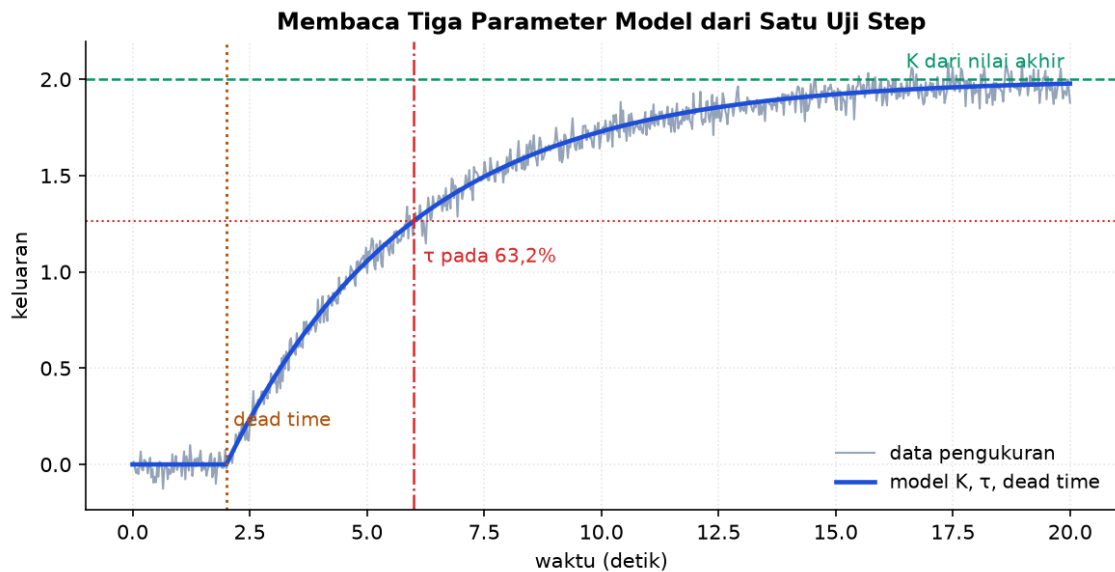
Model ruang keadaan menyatakan sistem sebagai sekumpulan persamaan diferensial orde satu yang saling terkait, dengan vektor keadaan sebagai ingatan sistem. Bentuk ini menampung sistem banyak masukan dan banyak keluaran secara alami, sedangkan fungsi transfer pada dasarnya menggambarkan satu pasangan masukan-keluaran.

Perbedaan yang lebih mendasar adalah bahwa ruang keadaan menampakkan seluruh dinamika internal, termasuk yang tidak terlihat dari keluaran. Dua sistem dapat memiliki fungsi transfer identik namun ruang keadaan berbeda; yang satu bisa memiliki mode internal tidak stabil yang tersembunyi, dan perbedaan itu menentukan apakah sistem aman dipakai.

Perpindahan antar keduanya bersifat searah tanpa kehilangan pada satu arah dan berisiko kehilangan pada arah lain. Dari ruang keadaan ke fungsi transfer selalu dapat dilakukan dengan rumus baku. Dari fungsi transfer ke ruang keadaan menghasilkan realisasi yang tidak tunggal, dan realisasi minimum hanya memuat bagian yang terkendali sekaligus teramati.

Untuk keperluan simulasi, bentuk ruang keadaan biasanya lebih disukai karena solver numerik bekerja langsung atas sistem persamaan orde satu. Integrasi dilakukan atas vektor keadaan, dan keluaran dihitung dari keadaan tersebut pada setiap langkah. Hubungan ini dirangkum dalam Persamaan (1).

$$\mathbf{x}' = \mathbf{A} \cdot \mathbf{x} + \mathbf{B} \cdot \mathbf{u}, \quad \mathbf{y} = \mathbf{C} \cdot \mathbf{x} + \mathbf{D} \cdot \mathbf{u} \quad | \quad \mathbf{G}(s) = \mathbf{C} \cdot (s\mathbf{I} - \mathbf{A})^{-1} \cdot \mathbf{B} + \mathbf{D} \quad (1)$$



Gambar 2. Penentuan gain statik, konstanta waktu, dan dead time dari satu uji step.

Gambar 2 menunjukkan bahwa ketiga parameter model orde satu dapat dibaca dari satu percobaan sederhana, dan justru kesederhanaan itulah yang membuatnya banyak dipakai di lapangan.

Solver Numerik dan Perangkat Kekakuan

Solver numerik memperkirakan penyelesaian dengan melangkah maju dalam waktu. Metode Euler maju paling sederhana namun paling tidak akurat dan paling mudah kehilangan kestabilan numerik. Runge-Kutta orde empat memberi ketepatan jauh lebih baik untuk biaya hitung yang masih wajar, dan menjadi pilihan baku pada banyak persoalan kontrol.

Kekakuan muncul ketika sistem memuat mode yang lajunya berbeda jauh, misalnya dinamika listrik dalam milidetik bersama dinamika termal dalam menit. Solver eksplisit terpaksa memakai langkah sekecil mode tercepat meskipun yang menarik hanya mode lambat, sehingga simulasi menjadi sangat lambat. Solver implisit dirancang untuk keadaan ini dan tetap stabil pada langkah besar.

Langkah waktu yang terlalu besar tidak selalu memberi hasil yang tampak salah. Bahayanya justru pada hasil yang terlihat masuk akal namun keliru, misalnya osilasi yang teredam padahal sistem sebenarnya tidak stabil. Karena itu pemeriksaan wajib dilakukan dengan mengulang simulasi pada langkah setengahnya; bila hasilnya berubah bermakna, langkah semula terlalu besar.

Pada simulasi controller digital, langkah waktu solver harus dibedakan dari waktu sampling controller. Plant berjalan kontinu sedangkan controller memperbarui keluarannya secara

berkala, dan pemodelan yang menyamakan keduanya akan menyembunyikan pengaruh waktu sampling terhadap kestabilan. Bentuk ringkasnya dituliskan pada Persamaan (2).

$$dt \leq 0,1 \cdot T_{\text{tercepat}} \quad | \quad \text{kekakuan} = T_{\text{lambat}}/T_{\text{cepat}} \gg 1 \quad (2)$$

Validasi Model Terhadap Data Nyata

Model belum berguna sebelum dibandingkan dengan perilaku perangkat sesungguhnya. Prosedur yang lazim memisahkan data menjadi bagian untuk identifikasi dan bagian untuk pengujian. Model disetel memakai bagian pertama, lalu dinilai memakai bagian kedua yang belum pernah dilihat.

Model yang cocok sempurna pada data identifikasi namun buruk pada data pengujian merupakan gejala terlalu banyak parameter. Menambah parameter selalu memperbaiki kecocokan pada data yang dipakai menyetelnya, dan perbaikan itu menyesatkan. Kriteria pemilihan model yang baik memberi hukuman terhadap jumlah parameter agar kecenderungan tersebut terkendali.

Metrik kecocokan perlu dipilih sesuai kepentingan. Galat kuadrat rata-rata memberi bobot besar pada penyimpangan besar dan cocok bila lonjakan berbahaya. Galat mutlak rata-rata lebih tahan terhadap pencilan. Untuk keperluan kontrol, kecocokan pada rentang frekuensi kerja controller jauh lebih penting daripada kecocokan rata-rata di seluruh rentang.

Perbedaan yang tersisa antara model dan perangkat bukan kegagalan melainkan informasi. Selisih itu perlu dinyatakan sebagai rentang ketidakpastian, dan controller dirancang agar tetap memenuhi spesifikasi di seluruh rentang tersebut, bukan hanya pada nilai nominalnya. Persamaan (3) memadatkan aturan tersebut.

$$RMSE = \sqrt{\text{mean}((y_{\text{ukur}} - y_{\text{model}})^2)} \quad | \quad \text{pisahkan data latih dan data uji} \quad (3)$$

Dari Simulasi ke Perangkat Keras Secara Bertahap

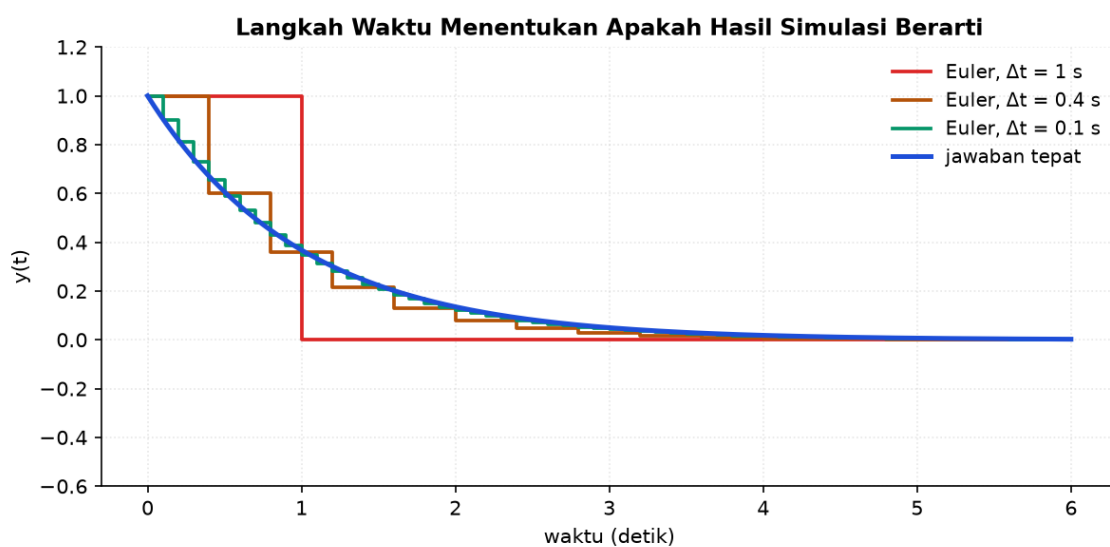
Pengujian bertahap mengurangi risiko dengan menambahkan satu unsur nyata pada satu waktu. Tahap pertama menjalankan seluruhnya di komputer. Tahap berikutnya menjalankan controller pada perangkat sasaran yang sesungguhnya sementara plant masih berupa model, sehingga pengaruh waktu sampling, aritmetika terbatas, dan jeda perhitungan mulai terlihat.

Tahap selanjutnya memakai plant nyata dengan pengaman tambahan berupa pembatasan keluaran, batas darurat, dan kemampuan mengembalikan kendali ke operator seketika. Setiap tahap memiliki kriteria lulus sendiri, dan kegagalan pada satu tahap mengembalikan pekerjaan ke tahap sebelumnya alih-alih dipaksakan maju.

Perbedaan yang paling sering mengejutkan pada peralihan ke perangkat keras adalah derau pengukuran. Sinyal yang bersih di simulasi menjadi bergetar di lapangan, dan aksi turunan yang tampak menguntungkan di komputer dapat membuat aktuator bergetar terus-menerus. Filter pada jalur turunan hampir selalu diperlukan.

Catatan pengujian setiap tahap merupakan bagian dari hasil kerja, bukan pelengkap. Ketika sistem bermasalah di kemudian hari, catatan itulah yang memungkinkan perbedaan antara masalah baru dan perilaku yang sejak awal memang sudah ada namun belum pernah menjadi persoalan.

simulasi murni → controller nyata + plant model → plant nyata terbatas → operasi



Gambar 3. Pengaruh langkah waktu integrasi terhadap hasil simulasi sistem yang sama.

Gambar 3 memuat peringatan penting: simulasi selalu menghasilkan angka, termasuk ketika langkah waktunya terlalu besar, sehingga kesalahan jenis ini tidak memberi tanda selain hasil yang keliru.

Mengidentifikasi Parameter dari Uji yang Sederhana

Model tanpa angka tidak berguna, dan angka itu paling mudah diperoleh dari uji step pada titik kerja normal. Katup atau masukan lain diubah sedikit, keluarannya direkam, lalu tiga parameter ditarik dari kurva yang terbentuk.

Gain statik diperoleh dari perbandingan perubahan keluaran terhadap perubahan masukan setelah keduanya menetap. Perubahan yang dipakai harus cukup besar untuk terbaca di atas derau, namun cukup kecil agar sistem tetap berada di daerah yang linier. Kedua tuntutan itu saling berlawanan, dan menyeimbangkannya adalah bagian dari keterampilan.

Konstanta waktu diperoleh dari waktu mencapai 63,2 persen perubahan akhir, dihitung sejak keluaran mulai bergerak. Dead time diperoleh dari jeda antara saat masukan diubah

dan saat keluaran mulai bergerak. Membedakan keduanya menuntut kecermatan, karena derau membuat titik mulai bergerak menjadi kabur.

Uji sebaiknya diulang beberapa kali, ke arah naik maupun turun. Bila hasilnya berbeda bermakna antara arah naik dan turun, sistem memiliki histeresis atau daerah mati yang perlu dimodelkan tersendiri; bila hasilnya berbeda antarpengulangan, derau atau gangguan luar terlalu besar sehingga uji perlu dirancang ulang.

uji step: K dari perbandingan akhir, τ dari 63,2 persen, L dari jeda awal

Linearisasi: Menjinakkan Nonlinieritas di Titik Kerja

Sebagian besar proses bersifat nonlinier, sementara hampir seluruh perkakas analisis mengandaikan linearitas. Jembatan di antara keduanya adalah linearisasi, yaitu mengganti hubungan nonlinier dengan garis singgungnya di titik kerja.

Secara matematis, fungsi nonlinier diuraikan di sekitar titik kerja dan hanya suku orde pertamanya yang dipertahankan. Turunan parsial terhadap tiap variabel menjadi koefisien model linier yang dihasilkan. Karena itu model linier selalu berbicara tentang simpangan dari titik kerja, bukan tentang nilai mutlak.

Kekeliruan yang sering terjadi adalah lupa bahwa variabelnya kini simpangan. Setpoint nol pada model linier berarti sistem berada tepat di titik kerja, bukan bahwa temperaturnya nol derajat. Menukar keduanya menghasilkan kesimpulan yang tampak masuk akal namun sepenuhnya keliru.

Jangkauan keberlakuannya perlu dinyatakan. Untuk hubungan yang lengkungannya landai, model linier dapat sah pada rentang lebar; untuk hubungan yang tajam, rentangnya sempit. Menguji seberapa jauh model masih memadai dilakukan dengan membandingkan keluarannya terhadap model nonlinier penuh pada beberapa jarak dari titik kerja. Rangkuman kuantitatifnya tertulis pada Persamaan (4).

$$dx' = A \cdot dx + B \cdot du \text{ dengan } A \text{ dan } B \text{ turunan parsial di titik kerja} \quad (4)$$

Memodelkan Gangguan dan Derau

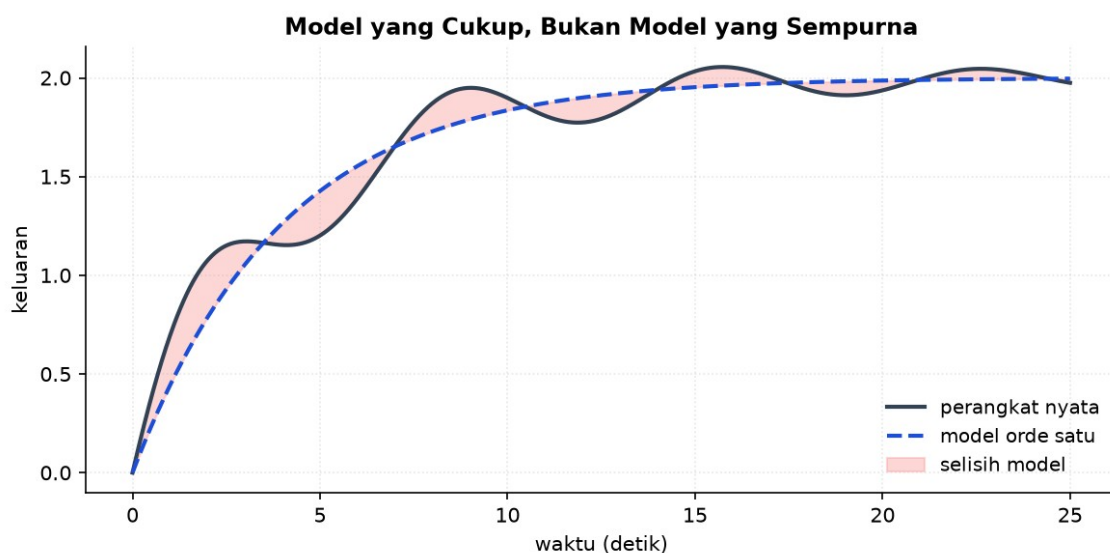
Model yang hanya memuat hubungan masukan-keluaran belum memadai untuk menilai rancangan kendali, karena sebagian besar pekerjaan controller justru menghadapi hal yang tidak dikehendaki: gangguan beban dan derau pengukuran.

Gangguan beban umumnya berfrekuensi rendah dan masuk di sisi masukan plant. Pemodelan yang lazim memakai step atau ramp untuk mewakili perubahan beban, ditambah komponen acak berfrekuensi rendah bila bebannya memang berfluktuasi. Yang penting bukan meniru bentuk sesungguhnya melainkan mewakili rentang frekuensinya.

Derau pengukuran berperilaku berlawanan: berfrekuensi tinggi dan masuk di jalur umpan balik. Pemodelan yang memadai cukup berupa komponen acak dengan simpangan baku sebesar derau sensor yang sebenarnya, yang dapat diukur langsung dengan merekam pembacaan saat proses diam.

Membedakan keduanya menentukan cara menanganinya. Gangguan berfrekuensi rendah ditolak dengan menaikkan gain loop pada frekuensi rendah, sementara derau berfrekuensi tinggi ditangani dengan menurunkan gain pada frekuensi tinggi. Karena keduanya berada di rentang berbeda, keduanya dapat ditangani sekaligus, dan itulah yang membuat pemodelan keduanya berguna.

gangguan: frekuensi rendah di masukan plant | derau: frekuensi tinggi di umpan balik



Gambar 4. Selisih antara perangkat nyata dan model sederhana yang tetap layak dipakai merancang.

Gambar 4 menegaskan bahwa selisih model bukan alasan menolak model: yang perlu dijawab adalah apakah selisih itu masih tertutup oleh margin rancangan.

Menyimpan Hasil agar Dapat Diulang Orang Lain

Simulasi yang tidak dapat diulang tidak dapat diperiksa, dan yang tidak dapat diperiksa tidak layak menjadi dasar keputusan. Karena itu penyimpanan hasil merupakan bagian dari pekerjaan, bukan tambahan setelahnya.

Yang wajib tersimpan bersama grafik adalah versi kode, seluruh nilai parameter, kondisi awal, jenis solver, langkah waktu, dan definisi masukan uji. Enam hal itu cukup bagi orang lain memperoleh angka yang sama, dan tanpa salah satunya pengulangan menjadi tebak-tebakan.

Praktik yang membantu adalah menyimpan parameter di berkas terpisah dari kode, bukan menuliskannya di tengah program. Dengan begitu satu berkas parameter dapat disimpan berdampingan dengan hasilnya, dan perbedaan antarpercobaan terbaca langsung tanpa membandingkan kode.

Penamaan yang tertib melengkapinya. Menyertakan tanggal dan pokok perubahan pada nama berkas hasil membuat riwayat percobaan terbaca berbulan-bulan kemudian, ketika ingatan tentang alasan tiap percobaan sudah lama hilang.

simpan: versi kode, parameter, kondisi awal, solver, dt, masukan uji

MENURUNKAN RUANG KEADAAN SISTEM MASSA-PEGAS-PEREDAM

Penurunan berikut memperlihatkan cara mengubah satu persamaan orde dua menjadi dua persamaan orde satu yang siap diintegrasikan solver numerik. Hubungan ini dirangkum dalam Persamaan (5).

1. Hukum Newton

$$m \cdot x'' + b \cdot x' + k \cdot x = F(t) \quad (5)$$

Jumlah gaya pada massa sama dengan massa dikali percepatan. Bentuk ringkasnya dituliskan pada Persamaan (6).

2. Pilih variabel keadaan

$$x_1 = x \text{ (posisi)}, \quad x_2 = x' \text{ (kecepatan)} \quad (6)$$

Dipilih besaran yang menyimpan energi: potensial pada pegas, kinetik pada massa. Persamaan (7) memadatkan aturan tersebut.

3. Persamaan pertama

$$x_1' = x_2 \quad (7)$$

Definisi langsung: turunan posisi adalah kecepatan. Rangkuman kuantitatifnya tertulis pada Persamaan (8).

4. Persamaan kedua

$$x_2' = (F - b \cdot x_2 - k \cdot x_1)/m \quad (8)$$

Diperoleh dengan menyelesaikan hukum Newton terhadap percepatan. Hubungan ini dirangkum dalam Persamaan (9).

5. Susun bentuk matriks

$$A = \begin{bmatrix} 0 & 1 \\ -k/m & -b/m \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} \quad (9)$$

Keluaran posisi memberi $C = [1, 0]$ dan $D = [0]$. Bentuk ringkasnya dituliskan pada Persamaan (10).

6. Periksa lewat pole

$$\det(sI - A) = s^2 + (b/m)s + k/m \quad (10)$$

Polinomial karakteristik sama dengan penyebut fungsi transfernya, sebagaimana seharusnya.

Bentuk ini siap diserahkan ke solver numerik karena hanya memuat turunan pertama. Dari sini pula terbaca bahwa rasio redaman adalah b dibagi dua akar km , dan frekuensi alami adalah akar k per m . Persamaan (11) memadatkan aturan tersebut.

CONTOH TERHITUNG: MEMILIH LANGKAH WAKTU SIMULASI

Diketahui: Sistem massa-pegas-peredam dengan $m = 2$ kg, $b = 12$ N·s/m, $k = 50$ N/m. Simulasi memakai solver langkah tetap

1. Hitung frekuensi alami

$$\omega_n = \sqrt{k/m} = \sqrt{50/2} = 5 \text{ rad/s} \quad (11)$$

Menentukan skala waktu dasar sistem. Rangkuman kuantitatifnya tertulis pada Persamaan (12).

2. Hitung rasio redaman

$$z = b/(2\sqrt{k\cdot m}) = 12/(2\sqrt{100}) = 0,6 \quad (12)$$

Nilai di antara nol dan satu, jadi respons berosilasi teredam. Hubungan ini dirangkum dalam Persamaan (13).

3. Cari pole

$$s = -z\omega_n \pm j\omega_n\sqrt{1-z^2} = -3 \pm j4 \quad (13)$$

Bagian nyata -3 memberi konstanta waktu peluruhan 1/3 detik. Bentuk ringkasnya dituliskan pada Persamaan (14).

4. Tentukan mode tercepat

$$T_{\text{tercepat}} = 1/3 = 0,333 \text{ s} \quad (14)$$

Hanya ada satu pasangan pole sehingga inilah skala tercepat. Persamaan (15) memadatkan aturan tersebut.

5. Pilih langkah waktu

$$\Delta t \leq 0,1 \cdot 0,333 = 0,033 \text{ s} \quad (15)$$

Diambil $dt = 0,01$ s untuk margin, sekitar sepertiga batas. Rangkuman kuantitatifnya tertulis pada Persamaan (16).

6. Verifikasi

ulangi pada $dt = 0,005$ s (16)

Bila puncak overshoot dan waktu menetap tidak berubah bermakna, $dt = 0,01$ s memadai.

Jawaban. Langkah waktu 0,01 detik memadai dan terbukti lewat pengulangan pada setengahnya. Bila kemudian ditambahkan dinamika actuator dengan konstanta waktu 5 milidetik, sistem menjadi kaku dan langkah harus turun ke 0,0005 detik atau solver diganti ke jenis implisit.

SALAH KAPRAH YANG SERING TERJADI

- Menyimpulkan dari satu simulasi tanpa memeriksa langkah waktu — Hasil yang salah karena langkah terlalu besar sering tetap terlihat wajar. Pengulangan pada setengah langkah adalah pemeriksaan termurah yang tersedia.
- Menilai model dari data yang dipakai menyetelnya — Kecocokan pada data identifikasi selalu membaik saat parameter ditambah. Hanya data uji yang belum pernah dilihat yang memberi penilaian jujur.
- Menyamakan langkah solver dengan waktu sampling controller — Plant berjalan kontinu, controller memperbarui berkala. Menyamakannya menyembunyikan pengaruh waktu sampling terhadap kestabilan.
- Mengabaikan mode internal saat memakai fungsi transfer — Dua sistem berfungsi transfer sama dapat berbeda keamanannya bila salah satunya menyimpan mode tidak stabil yang tidak teramati.
- Membandingkan simulasi dan pengukuran pada titik kerja berbeda — Selisihnya akan tampak seperti kesalahan model padahal hanya akibat linearisasi di titik yang berlainan.

DAFTAR PERIKSA SEBELUM LANJUT

- ☐ Pertanyaan yang hendak dijawab simulasi dinyatakan lebih dulu
- ☐ Seluruh parameter, kondisi awal, dan asumsi tercatat
- ☐ Masukan uji dipilih sesuai sifat yang diperiksa
- ☐ Langkah waktu diverifikasi dengan pengulangan pada setengahnya

- ☐ Kekakuan sistem diperiksa sebelum memilih solver
- ☐ Model diuji pada data yang tidak dipakai menyetelnya
- ☐ Ketidakpastian model dinyatakan sebagai rentang, bukan diabaikan
- ☐ Rencana pengujian bertahap ke perangkat keras sudah disusun

TUGAS

Tugas dikerjakan pada halaman modul interaktif agar penilaiannya otomatis. Bobotnya 50 poin: sepuluh soal pilihan ganda @1 poin, sepuluh soal komputasi tingkat dasar-menengah @2 poin, dan lima soal komputasi tingkat lanjut @4 poin. Soal komputasi dikerjakan dengan Python; yang dinilai adalah nilai yang dicetak, bukan cara penulisan programnya.

A. Pilihan Ganda (10 soal @1 poin)

1. Simulasi yang berguna dirancang seperti eksperimen. Unsur yang paling sering terlupakan namun menentukan apakah hasilnya dapat dipercaya adalah...
 - A. Warna dan tata letak grafik keluaran
 - B. Solver beserta langkah waktu yang dipakai
 - C. Nama berkas tempat hasil disimpan
 - D. Jumlah titik data yang ditampilkan pada grafik
2. Sistem massa-pegas-peredam dapat berosilasi sedangkan sistem termal orde satu tidak, karena...
 - A. Sistem mekanik selalu memiliki gain lebih besar
 - B. Sistem termal selalu memiliki dead time
 - C. Sistem mekanik menyimpan energi dalam dua bentuk sekaligus, kinetik dan potensial
 - D. Sistem termal tidak memiliki redaman
3. Model ruang keadaan lebih unggul daripada fungsi transfer terutama karena...
 - A. Menampakkan seluruh dinamika internal, termasuk yang tidak terlihat dari keluaran
 - B. Selalu menghasilkan perhitungan yang lebih cepat
 - C. Tidak memerlukan kondisi awal
 - D. Hanya berlaku untuk sistem satu masukan satu keluaran
4. Sistem disebut kaku (stiff) apabila...

- A. Konstanta pegasnya sangat besar
 - B. Memuat mode dengan laju yang berbeda jauh, sehingga solver eksplisit terpaksa memakai langkah sangat kecil
 - C. Responsnya tidak pernah menetap
 - D. Memiliki lebih dari tiga variabel keadaan
5. Cara termurah memeriksa apakah langkah waktu simulasi sudah memadai adalah...
- A. Menambah jumlah titik pada grafik
 - B. Mengganti solver ke jenis lain lalu membandingkan waktu komputasi
 - C. Mengulang simulasi pada setengah langkah dan memastikan hasilnya tidak berubah bermakna
 - D. Memperpanjang durasi simulasi
6. Model yang cocok sempurna pada data identifikasi namun buruk pada data pengujian menunjukkan...
- A. Sensor yang dipakai rusak
 - B. Terlalu banyak parameter sehingga model menghafal derau alih-alih menangkap dinamika
 - C. Langkah waktu simulasi terlalu kecil
 - D. Sistem sebenarnya linier
7. Pada pengujian bertahap menuju perangkat keras, tahap yang menjalankan controller nyata terhadap plant model berguna untuk menangkap...
- A. Kesalahan pemilihan konstanta pegas
 - B. Gangguan lingkungan di lapangan
 - C. Keausan mekanis jangka panjang
 - D. Pengaruh waktu sampling, aritmetika terbatas, dan jeda perhitungan
8. Membandingkan hasil simulasi dengan data pengukuran pada titik kerja yang berbeda berisiko karena...
- A. Selisihnya akan tampak seperti kesalahan model padahal hanya akibat linearisasi di titik berlainan
 - B. Data pengukuran selalu lebih akurat daripada simulasi

- C. Simulasi tidak dapat menerima masukan berupa data terukur
- D. Titik kerja tidak memengaruhi parameter sistem linier
9. Metode Runge-Kutta orde empat lebih disukai daripada Euler maju terutama karena...
- A. Selalu lebih cepat dihitung
- B. Tidak memerlukan penentuan langkah waktu
- C. Ketepatannya jauh lebih baik untuk biaya hitung yang masih wajar
- D. Dapat menangani sistem nonlinier sedangkan Euler tidak
10. Ketidakpastian yang tersisa antara model dan perangkat sebaiknya diperlakukan sebagai...
- A. Kegagalan pemodelan yang harus dihilangkan sepenuhnya
- B. Rentang yang harus tetap dipenuhi spesifikasi oleh rancangan controller
- C. Hal yang diabaikan karena controller akan menanganinya sendiri
- D. Alasan untuk menambah parameter model sampai selisihnya nol

B. Komputasi Dasar-Menengah (10 soal @2 poin)

Parameter acuan: Sistem massa-pegas-peredam dengan persamaan $m \cdot x'' + b \cdot x' + k \cdot x = F(t)$, diberi gaya step $F = 10$ N dan kondisi awal nol. Dinamika actuator diabaikan kecuali soal menyebutkannya. Gunakan angka ini kecuali soal menyebut angka lain. Rinciannya dirangkum pada Tabel 1.

Tabel 1. Parameter acuan yang dipakai seluruh soal komputasi modul ini.

Parameter	Nilai
m	2 kg
b	12 N·s/m
k	50 N/m
F	10 N (step)

C1. Hitung frekuensi alami sistem acuan dalam rad/s. Cetak: ω_n .

– PETUNJUK: Langkah: 1) tulis $\omega_n = \sqrt{k/m}$. 2) masukkan $k = 50$ dan $m = 2$. 3) hitung akarnya.

C2. Hitung rasio redaman sistem acuan. Cetak: z .

– PETUNJUK: Langkah: 1) tulis $z = b/(2 \cdot \sqrt{k \cdot m})$. 2) hitung $k \cdot m$ lalu akarnya. 3) bagi b dengan dua kali hasil itu.

C3. Hitung frekuensi teredam dalam rad/s. Cetak: ω_d .

– PETUNJUK: Langkah: 1) peroleh ω_n . 2) peroleh z . 3) hitung $\sqrt{1 - z^2}$. 4) $\omega_d = \omega_n \times \sqrt{1 - z^2}$.

C4. Hitung perpindahan tunak massa akibat gaya step, dalam meter. Cetak: x_{ss} .

– PETUNJUK: Langkah: 1) pada keadaan tunak percepatan dan kecepatan bernilai nol. 2) persamaannya menyisakan $k \cdot x = F$. 3) $x_{ss} = F/k$.

C5. Hitung overshoot respons dalam persen. Cetak: M_p dalam persen.

– PETUNJUK: Langkah: 1) peroleh z . 2) hitung $\sqrt{1 - z^2}$. 3) hitung $\pi \cdot z / \sqrt{1 - z^2}$. 4) $M_p = \exp(-\text{hasil})$ lalu kalikan 100.

C6. Hitung perpindahan puncak massa dalam meter. Cetak: x_{puncak} .

– PETUNJUK: Langkah: 1) peroleh z . 2) hitung M_p . 3) hitung $x_{ss} = F/k$. 4) $x_{puncak} = x_{ss} \times (1 + M_p)$.

C7. Hitung waktu puncak dalam detik. Cetak: t_p .

– PETUNJUK: Langkah: 1) peroleh ω_n dan z . 2) hitung $\sqrt{1 - z^2}$. 3) $\omega_d = \omega_n \times \sqrt{1 - z^2}$. 4) $t_p = \pi / \omega_d$.

C8. Hitung waktu menetap pada pita dua persen, dalam detik. Cetak: t_s .

– PETUNJUK: Langkah: 1) peroleh ω_n dan z . 2) hitung hasil kali $z \times \omega_n$. 3) $t_s = 4 / (z \times \omega_n)$.

C9. Hitung konstanta waktu peluruhan selubung osilasi, dalam detik. Cetak: τ selubung.

– PETUNJUK: Langkah: 1) peroleh z dan ω_n . 2) bagian nyata pole adalah $-z \cdot \omega_n$. 3) konstanta waktu = $1 / (z \times \omega_n)$.

C10. Tentukan langkah waktu maksimum untuk solver langkah tetap, memakai aturan $dt \leq 0,1$ kali konstanta waktu tercepat. Cetak dalam detik.

– PETUNJUK: Langkah: 1) peroleh z dan ω_n . 2) hitung konstanta waktu tercepat $1 / (z \times \omega_n)$. 3) kalikan dengan 0,1.

C. Komputasi Lanjut (5 soal @4 poin)

C11 (Lanjut). Model dilengkapi dinamika actuator orde satu berkonstanta waktu 5 milidetik. Hitung rasio kekakuan sistem, yaitu konstanta waktu paling lambat dibagi yang paling cepat. Cetak: rasio kekakuan.

– PETUNJUK: Langkah: 1) peroleh ω_n . 2) peroleh z . 3) hitung $z \times \omega_n$. 4) konstanta waktu mekanik = $1/(z \times \omega_n)$. 5) kenali konstanta waktu aktuator 0,005 s sebagai yang tercepat. 6) bagi yang lambat dengan yang cepat.

C12 (Lanjut). Hitung energi potensial yang tersimpan di pegas pada saat perpindahan mencapai puncak, dalam joule. Cetak: energi (J).

– PETUNJUK: Langkah: 1) peroleh ω_n . 2) peroleh z . 3) hitung M_p . 4) hitung $x_{ss} = F/k$. 5) $x_{puncak} = x_{ss} \times (1 + M_p)$. 6) hitung energi $0,5 \times k \times x_{puncak}^2$.

C13 (Lanjut). Tentukan nilai koefisien redaman kritis, lalu hitung berapa besar b harus ditambah agar sistem menjadi teredam kritis. Cetak penambahannya dalam N·s/m.

– PETUNJUK: Langkah: 1) tulis syarat redaman kritis $z = 1$. 2) susun $z = b/(2 \cdot \sqrt{k \cdot m})$. 3) hitung $k \cdot m$. 4) hitung akarnya. 5) $b_{kritis} = 2 \times$ akar tersebut. 6) kurangkan b sekarang dari b_{kritis} .

C14 (Lanjut). Dengan peredam diganti menjadi teredam kritis, hitung selisih waktu menetap pita dua persen terhadap sistem semula, dalam detik. Cetak: selisih t_s .

– PETUNJUK: Langkah: 1) hitung t_s sistem semula dari z dan ω_n . 2) hitung b_{kritis} . 3) pada redaman kritis $z = 1$. 4) ω_n tidak berubah karena k dan m tetap. 5) hitung t_s kritis = $4/(z \times \omega_n)$. 6) ambil selisih t_s semula dikurangi t_s kritis.

C15 (Lanjut). Gaya step diperbesar menjadi 25 N sementara ruang gerak massa dibatasi 0,5 m. Hitung perpindahan puncak yang terjadi, dalam meter, lalu nilai apakah batas ruang terlampaui. Cetak: x_{puncak} .

– PETUNJUK: Langkah: 1) peroleh ω_n . 2) peroleh z . 3) hitung $\sqrt{1 - z^2}$. 4) hitung M_p . 5) hitung x_{ss} baru = $25/k$. 6) $x_{puncak} = x_{ss} \times (1 + M_p)$. 7) bandingkan dengan batas 0,5 m.

DAFTAR PUSTAKA

Ljung, L. (1999). System identification: Theory for the user (2nd ed.). Prentice Hall.

Seborg, D. E., Edgar, T. F., Mellichamp, D. A., & Doyle, F. J. (2016). Process dynamics and control (4th ed.). Wiley.

Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2019). Feedback control of dynamic systems (8th ed.). Pearson.

Harris, C. R., dkk. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.

Virtanen, P., dkk. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17(3), 261-272.